

IPL Dataset 2008-25

Analysis Report — Exploratory Data Analysis using Python & Pandas

1 Problem Statement

Background

You are working as a Data Analyst for a sports analytics firm. The firm has collected match-level data from the Indian Premier League (IPL) spanning from 2008 to 2025. The goal is to extract meaningful insights about team performance, player of the match records, venue statistics, and tournament dynamics to support analysts, commentators, and franchise strategists.

You are provided with a dataset (matches.csv) containing match-level records across all IPL seasons.

The dataset includes teams, venues, cities, seasons, match types, winners, result margins, player of the match, umpire details, and more.

IPL Dataset Questions to Answer

Your analysis should answer the following questions clearly.

1. Which team has won by the highest run margin in IPL so far?
2. Who is the most Player of the Match for Royal Challengers Bangalore?
3. Which team has won the most matches in RCB vs KKR?
4. Which player has been the most Player of the Match for each team in the IPL?
5. Which city has hosted the most IPL matches?
6. Which team won the final match in season 2007/2008?
7. Which umpire has officiated in the most matches so far?
8. Which team has arrived the most in the Final match?
9. Which team has reached the finals and won the most number of times?
10. Which team reaches Qualifier 1 more times?

2

Solution Overview

We will solve this problem using Exploratory Data Analysis (EDA).

EDA helps us understand patterns, relationships, and anomalies in the data before applying any advanced modeling.

The solution is divided into logical steps:

- Data loading and understanding
- Data cleaning and preparation
- Analysis to answer each business question
- Summary of insights

Each step below explains what we are doing and why.

3

Data Loading and Initial Understanding

Step 1: Import Required Libraries

We use Pandas for data manipulation and Matplotlib for visualization.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Step 2: Load the Dataset

Why this step matters:

- Allows us to inspect the structure of the dataset
- Helps verify column names and data types

```
df = pd.read_csv('matches.csv')
df.head()
```

Step 3: Basic Dataset Overview

Why this step matters:

- Identifies number of rows and columns
- Reveals Dataset Basic Information like Number of Rows and Columns, Column Names with Datatypes and NULL Values, Memory Usage
- Reveals missing values and incorrect data types

```
df.shape
df.info()
print(df.isna().sum())
```

4 Data Cleaning and Preparation

Real-world datasets are messy. Cleaning ensures accurate analysis.

Step 4: Standardize Column Names

Why: Makes column names consistent and easier to use in code.

```
df.columns = df.columns.str.strip().str.lower().str.replace(' ', '_')
print(df.columns.tolist())
```

Step 5: Fill Missing Values

Why: Prevents errors in groupby and comparisons.

```
df['city'] = df['city'].fillna('Unknown')
df['target_runs'] = df['target_runs'].fillna(0).astype(int)
df['result_margin'] = df['result_margin'].fillna(0).astype(int)
```

Step 6: Remove Duplicates

Why: Duplicate rows can skew averages and insights.

```
df.drop_duplicates(inplace=True)
```

5 Answering IPL Dataset with Analysis

Q1 Which team has won by the highest run margin in IPL so far?

Why: Identifies the most dominant victory by run difference across all IPL matches.

```
max_run_margin = df['result_margin'].max()
Team = df[df['result_margin'] == max_run_margin]['winner'].values[0]
print(f'Highest Run Margin Winner: {Team} with margin {max_run_margin}.')
```

Note: `.values[0]` is used to extract the scalar team name from the filtered DataFrame, since filtering returns a Series — not a single value.

Q2 Who is the most Player of the Match for Royal Challengers Bangalore?

Why: Highlights the most impactful player for RCB — a franchise-level performance insight.

```
rcb_pom = df[df['winner'] == 'Royal Challengers Bangalore']['player_of_match']
most_pom = rcb_pom.value_counts().idxmax()
print(f'Most Player of the Match for RCB is {most_pom}.')
```

Note: `.value_counts()` counts frequency of each player name; `.idxmax()` returns the player with the highest count.

Q3 Which team has won the most matches in RCB vs KKR?

Why: Head-to-head rivalry stats help franchise strategists understand historical dominance.

```
rcb_kkr = df[
    (df['team1']=='Royal Challengers Bangalore') & (df['team2']=='Kolkata Knight Riders') |
    (df['team1']=='Kolkata Knight Riders') & (df['team2']=='Royal Challengers Bangalore')
].groupby('winner').size()
most_wins = rcb_kkr.idxmax()
print(f'Most wins in RCB vs KKR is {most_wins}.')
```

Q4 Which player has been the most Player of the Match for each team?

Why: Team-level PoM leaders reveal individual players who drive franchise success.

```
Each_team_pom = df.groupby('winner')['player_of_match'].agg(
    lambda x: x.value_counts().idxmax()
)
# Scatter plot
plt.figure(figsize=(10, 4))
plt.scatter(Each_team_pom.index, Each_team_pom.values)
plt.xticks(rotation=60)
plt.xlabel('Teams')
plt.ylabel('Most Player of the Match')
plt.title('Most Player of the Match for Each Team in IPL')
plt.show()
```

Q5 Which city has hosted the most IPL matches?

Why: Venue and city hosting stats are important for broadcasting rights and franchise revenue analysis.

```
most_hosted_city = df['city'].value_counts().idxmax()
venue_name = df[df['city'] == most_hosted_city]['venue'].value_counts().idxmax()
print(f'Most hosted city: {most_hosted_city}, Venue: {venue_name}.')
```

Note: `.idxmax()` on `value_counts()` returns the label (city/venue name) with the maximum frequency — not the count itself.

Q6 Which team won the final match in season 2007/2008?

Why: Season-wise finals reveal the inaugural IPL champion — historically significant for franchise records.

```
final_match = df[df['season'] == '2007/08'].groupby('match_type').get_group('Final')
final_winner = final_match['winner'].iloc[0]
print(f'Winner of 2007/2008 Final: {final_winner}.')
```

Note: `.iloc[0]` selects the first row by integer position — since the Final is a single match, this safely extracts the winner.

Q7 Which umpire has officiated in the most matches so far?

Why: Umpire statistics help cricket boards evaluate experience and appointment frequency.

```
Umpire_matches1 = df['umpire1'].value_counts().idxmax()
Umpire_matches2 = df['umpire2'].value_counts().idxmax()
print(f'Most officiated Umpire1: {Umpire_matches1}.')
print(f'Most officiated Umpire2: {Umpire_matches2}.')
```

Q8 Which team has arrived the most in the Final match?

Why: Tracks consistency of franchises in reaching the biggest stage — reflects long-term squad quality.

```
final_matches = df[df['match_type'] == 'Final']
final_teams = pd.concat([final_matches['team1'], final_matches['team2']])
most_arrivals = final_teams.value_counts().idxmax()
print(f'Team with most Final appearances: {most_arrivals}.')
```

Q9 Which team has reached the finals and won the most number of times?

Why: Distinguishes teams that reach the Final vs those who consistently win it — championship pedigree.

```
final_matches = df[df['match_type'] == 'Final']['winner']
most_arrivals = final_matches.value_counts().idxmax()
print(f'Team with most Final wins: {most_arrivals}.')
```

Q10 Which team reaches Qualifier 1 more times?

Why: Qualifier 1 appearances indicate top-2 league stage finishes — a mark of consistent regular season performance.

```
Qualifier_1 = df[df['match_type'] == 'Qualifier 1']
Qualifier = pd.concat([Qualifier_1['team1'], Qualifier_1['team2']])
Highest_Qualifier_1 = Qualifier.value_counts().idxmax()
print(f'Team with most Qualifier 1 appearances: {Highest_Qualifier_1}.')
```

6 Deeper Insights — Chart Analysis

Question 4 (Chart): Most Player of the Match for Each Team

Why:

- Visual comparison reveals top performers per franchise
- Scatter plot with labeled points provides quick reference
- Helps franchise management identify which players deliver under pressure

```
plt.figure(figsize=(10, 4))
plt.scatter(Each_team_pom.index, Each_team_pom.values)
plt.xticks(rotation=60)
plt.xlabel('Teams')
plt.ylabel('Most Player of the Match')
plt.title('Most Player of the Match for Each Team in IPL')
plt.tight_layout()
plt.show()
```

Dataset Column Reference

Column Name	Data Type	Description
season	String/Int	IPL season year (e.g., 2008, 2007/08)
team1	String	First team in the match
team2	String	Second team in the match
city	String	City where match was played
venue	String	Stadium/ground name
match_type	String	Type of match (League, Final, Qualifier 1, etc.)
winner	String	Team that won the match
result_margin	Integer	Winning margin (runs or wickets)
player_of_match	String	Player awarded Man of the Match
umpire1	String	First on-field umpire
umpire2	String	Second on-field umpire
target_runs	Integer	Target set for second innings

From the analysis of the IPL 2008-25 matches dataset, we can conclude:

Highest Run Margin	The team that won by the highest run margin showcases the most dominant batting performance in IPL history.
Star Performers	A single player often wins multiple Player of the Match awards for their franchise — franchise icons drive results.
Head-to-Head Rivalry	RCB vs KKR head-to-head data reveals a consistent pattern of dominance by one side across seasons.
Venue Dominance	A handful of cities host a disproportionately large share of IPL matches — venue selection is driven by infrastructure and fan base.
Final Appearances	Teams that consistently appear in Finals reflect robust squad-building and coaching stability over multiple seasons.
Qualifier 1 Consistency	Top Qualifier 1 teams are synonymous with strong league stage performances — they set the standard every season.
Umpire Experience	The most officiated umpires have decades of consistency and trust from cricket boards.
Champion Pedigree	Teams with the most Final wins distinguish themselves not just as finalists but as genuine title contenders.

These insights can guide franchise management, auction strategies, and match-day tactics.

IPL Matches Dataset 2008–25 | Analysis powered by Python & Pandas | Data Analytics Report